

CPSC 304 Project Cover Page

Milestone #: 3

Date: 03/06/26

Group Number: 36

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Julian Wright	84895358	w6i9k	jwrigh13@student.ubc.ca
Dorson Tang	94262474	d5v6x	dtang08@student.ubc.ca
Krishna Garcha	40548323	g8a8b	garchakri@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Project Description

The domain of our project is electronics project management. The idea is to allow users to create projects and add electronic components (such as resistors, capacitors, etc.) for organizational/ tracking purposes. They can also choose suppliers for the components and track orders from those suppliers. Users can add locations (e.g., drawers, component bins, etc.) to both their components and projects to be able to find them better, something especially important for the components as they can get lost easily.

Timeline / task breakdown

Projection location - Dorson MAR 13

- Create a GET `"/locations?COLUMNS"` endpoint that returns the specified columns in the specified order
 - COLUMNS represent the order of the received columns
- Create a select multiple element that has the columns of the location relation. Users can select multiple columns in a specified order, then when done, will get to see the columns displayed in that order.
 - Other columns will not be present (wont be sent by backend)
 - Can structure request using JSON

Deleting components - Krishna Mar 20

- Components will be in a table format where each tuple entry can have a checkbox to select the tuple, and there will be a set of icons for possible actions (i.e., "delete" tuple)
- Call DELETE `"/components/{part_num}"` API endpoint on the specified component
- Refresh components table when call is goes through
- Frontend for deleting component
 - A delete button will be visible in each tuple entry for the component table
 - When selecting multiple tuples, a group delete of tuples can be performed
- Backend for querying and deleting components
 - Create a DELETE `"/component/{part_num}"` endpoint to delete components

- Be careful to only delete what's necessary, and to consider specified delete behaviour (e.g., cascading)
- When location is deleted, set location to NULL in the component
- Deleting the supplier will not do anything - the supplier won't be deleted and the attribute stays in component
- Deleting the IsA child requires deleting the related component
- Able to delete multiple components

Join components and location - Dorson MAR 20

- Allows users to locate the label of the location where the component is stored (since WHERE has to be defined, we can do something like find the label of the location where resistors with more than 500 resistance or something like that)
- Create a GET "/storages?WHERE" endpoint that returns the JOIN product of components and location
 - User can define WHERE, so we can add query parameters to the route
- Create multiple select dropdowns that define the desired column, the operator (>, =, etc), and the value
 - Once queried, we return the query in a table

Inserting into orders - Julian Mar 20

- Frontend for rendering orders
 - Create a table with the right columns for orders
 - Call GET "/orders" API endpoint that returns all orders
 - Render an order per row
- Frontend for creating an order
 - Button on the main components display to create an order
 - Modal w/ form for entering order information
 - Confirm button calls POST "/order" API endpoint with data in body
 - Refresh orders table
- Backend for querying and inserting orders
 - Create GET "/orders" endpoint that returns all orders from the DB
 - Create POST "/order" endpoint that inserts order into table given user-provided values

Updating locations - Julian Mar 24

- Frontend for rendering locations
 - Create a location element that's a grid
 - Add a location element per location
 - Assign each grid cell a name associated with the "position" field on Location
 - Render projects/components in the grid cells
- Frontend for editing location
 - Should be a button per location element for updating its name ("storage_name") and facility ("facility")
 - Within location element config, should allow editing names of grid cells ("position") and assigning a project/component to that cell
- Backend for querying and updating locations
 - Create a GET "/locations" API endpoint that returns all locations
 - Create a PUT "/location/{facility}/{storage_name}/{position}" endpoint that updates fields on element
 - Need to beware of changing any of these fields because the endpoint would change for that location, need to keep track well in frontend

Selecting components - Krishna Mar 24

- Frontend for selecting component
 - Add a filter button to the components table that will bring up a dialog
 - The user can add rows to the dialog, where each row has 3 dropdowns (AND/OR, column, operation) and a text field for an input value
 - Dialog has an accept and cancel button - cancel closes dialog and accept sends request to refresh components list
 - Calling GET endpoint "/components" with query parameters for filters using some specified format (e.g., "?quantity=<10&price=|>=15")
 - Another option would be to use the request body and represent the query using JSON
 - Refresh the components table upon fetch
- Backend for selecting components

- Create the GET “component?<key>=<op><value>...” endpoint, which takes a set of predicates as query parameters and returns only rows that satisfy those predicates

Division (may be hardcoded) - Dorson MAR 27

- Create a GET “/locations?has-all=“resistor” route to return all of the locations that have all resistors (or something similar)
- Create a division button that does this predefined division to find the locations that have all resistors

Aggregation with GROUP BY (may be hardcoded) - Julian Mar 28

- *Count number of components per supplier in orders*
- Add a button to an order entry (however those are rendered, probably a table) that will pop up with the answer
 - Table of supplier-count

Aggregation with HAVING (may be hardcoded) - Krishna Mar 28

- Perform the aggregate function on the groups of tuples given by GROUP BY
- GROUP BY components by their storage_name, and perform HAVING that checks if the max quantity is < 25
- SELECT storage_name, MAX(quantity)
FROM component
GROUP BY storage_name
HAVING max(quantity) < 25
- Add a “HAVING” button to filter for this particular query, the importance of the query being for the user to check if they may need to restock

Nested aggregation with GROUP BY (may be hardcoded) - Julian Mar 28

- Get price of all orders less than some budget that have at least one component
- SELECT p1.order_number, SUM(component.price * component.quantity)
FROM component FULL JOIN purchaseincludes p1 ON
component.part_num = p1.part_num GROUP BY p1.order_number
HAVING SUM(component.price * component.quantity) < **\$BUDGET** AND 1
<= (SELECT COUNT(*) FROM purchaseincludes p2 WHERE
p2.order_number = p1.order_number);

- Add text budget input, button to filter table

Miscellaneous front end maintenance - Dorson MAR 31

- Sanitize any inputs to disallow SQL injection
 - Would most likely be server side backend
- Ensure the front end is intuitive to non-cs users
- Handle errors such as duplicate inserts or invalid characters
 - Most likely with notifications below with exact message why it was unsuccessful
- Add notifications that signal to users if modifications were successful or not
 - Potentially use react-toastify

Milestone 4: Implementation due - April 2nd, 10PM

AI Tool Use Disclaimer

No AI tools were used for this milestone.